

Basic Abstractions in Distributed Algorithms II

Based on: Cachin, Guerraoui, Rodrigues

Danilo Kačanski

Faculty of Technical Sciences, Novi Sad
Applied algorithms in Control Systems

April 2026.

Failure Models

Failure Models Comparison

Model	What can go wrong?
No Failure	All processes behave correctly
Crash-Stop	Process halts permanently
Omission	Process may drop messages
Byzantine	Process can behave arbitrarily

Key Point

Each stronger model breaks **more assumptions** about process behavior.

Failure Model Interface (failures/crash.go)

FailureInjector Interface

```
type FailureInjector interface {  
    IsAlive(pid process.ProcessID) bool  
    ShouldDeliver(msg process.Message) bool  
    MaybeAlter(from process.ProcessID,  
        msgs []process.Message, rng *rand.Rand) []process.Message  
    CrashProcess(pid process.ProcessID)  
    RecoverProcess(pid process.ProcessID)  
}
```

Key Idea

Failure models are used by the **runtime** to simulate faults, while the algorithm itself remains unchanged.

Key Operations

- `IsAlive(pid)`: checks if a process can receive messages
- `ShouldDeliver(msg)`: decides if a message is dropped or delivered
- `MaybeAlter(...)`: allows modification of outgoing messages for Byzantine processes
- `CrashProcess(pid)`: simulates a crash
- `RecoverProcess(pid)`: simulates recovery

No Failure Model (failures/crash.go)

- The simplest model: **no faults occur**.
- Used as a **baseline execution**.

Behavior

- all processes are always alive
- all messages are always delivered
- messages are never modified

Key Idea

Provides a clean environment to understand the system before introducing failures.

No Failure Key Operations

Key Operations

- `IsAlive(pid)`: always returns true
- `ShouldDeliver(msg)`: always returns true
- `MaybeAlter(...)`: returns messages unchanged
- `CrashProcess(pid)`: no effect
- `RecoverProcess(pid)`: no effect

Crash-Stop Failure Model (failures/crash.go)

- Models the simplest type of failure: **process crash**.
- A crashed process:
 - stops executing completely
 - does not send messages
 - does not receive messages

Properties (Cachin et al.)

- once crashed $\Rightarrow$ **stops forever**
- no malicious behavior (no corrupted messages)
- no recovery in crash-stop model

Key Idea

Crash failures model systems where processes may **fail silently**, but never behave incorrectly.

Crash Failure Key Operations

Key Operations

- `IsAlive(pid)`: returns false if the process has crashed
- `ShouldDeliver(msg)`: always returns true, because delivery is controlled by `IsAlive`
- `MaybeAlter(...)`: messages are never modified
- `CrashProcess(pid)`: marks process as permanently crashed
- `RecoverProcess(pid)`: does nothing (no recovery)

Behavior in Simulator

- once a process crashes:
 - it stops receiving messages
 - it stops producing new messages
- the rest of the system continues executing normally

Omission Failure Model (failures/omission.go)

- Models failures where **messages are silently lost**.
- The process itself is still **running normally**.
- Unlike crashes:
 - process does NOT stop
 - but some messages are never received

Types of Omission

- **Send omission:** process fails to send a message
- **Receive omission:** process fails to receive a message

Key Idea

Omission failures are subtle: processes appear correct, but **communication is unreliable at the process level**.

Omission Failure Key Operations

Key Operations

- `IsAlive(pid)`: always returns true
- `ShouldDeliverRng(msg, rng)`: may return false based on omission probability
- `MaybeAlter(...)`: messages are never modified
- `CrashProcess(pid)`: no effect
- `RecoverProcess(pid)`: no effect

Behavior in Simulator

- each process can have its own **omission rate**
- messages addressed to that process may be: delivered normally or silently dropped
- harder to detect than crashes:
 - process is alive but behaves inconsistently

Byzantine Failure Model (failures/byzantine.go)

- Models the **strongest and most general** type of failure.
- A Byzantine process can behave **arbitrarily**.
- They are not limited to crashing or dropping messages.

Possible Behaviors

- send incorrect or unexpected messages
- modify message content (corruption)
- impersonate other processes (without authentication)
- omit messages or behave inconsistently
- combine all behaviors unpredictably

Key Idea

Byzantine failures model **malicious or unpredictable behavior**, not just accidental faults.

Byzantine Failure Key Operations

Key Operations

- `IsAlive(pid)`: always returns true
- `ShouldDeliver(msg)`: always returns true
- `MaybeAlter(from, msgs, rng)`:
 - applies arbitrary modifications to outgoing messages only for processes marked as Byzantine
- `CrashProcess(pid)`: no effect
- `RecoverProcess(pid)`: no effect

Behavior in Simulator

- selected processes are marked as **Byzantine**
- their outgoing messages can be: modified, replaced, dropped, arbitrarily generated
- behavior is controlled by a custom **alteration function**

Why Failure Models Matter

- Distributed algorithms must tolerate **faults**.
- The **failure model** defines what kinds of faults the algorithm must handle.
- Stronger failure models require **more complex algorithms**.

Key Insight

The choice of a failure model determines **what correctness means** for a distributed algorithm.

Cryptographic Primitives

Why is Cryptography needed in Distributed Systems?

- Distributed systems often operate in **untrusted environments** such as:
 - open networks (Internet)
 - multiple independent participants
- Messages can be:
 - intercepted
 - modified
 - forged
- We need mechanisms to ensure **secure communication**

Motivation

Without cryptography, a process cannot distinguish between a **valid message** and a **malicious one**.

What does Cryptography provide?

- **Integrity**
 - detect if a message was modified in transit
- **Authentication**
 - verify the identity of the sender
- **Non-repudiation**
 - sender cannot deny having sent the message

Key Idea

Cryptography allows processes to **trust messages**, even when the network or other processes cannot be trusted.

Where does Cryptography fit in our architecture?

- Cryptography is applied **before messages are sent**
- And verified **before messages are delivered**
- It acts as a layer between:
 - **process logic** (what we want to send)
 - **network/link** (how it is transmitted)

Architecture View

process → crypto → link → crypto → process

Interpretation

- Sender: attaches cryptographic proof (e.g., MAC/signature)
- Receiver: verifies the proof before accepting the message
- Any tampering is **detected and rejected**

Hash Functions (crypto/hash.go)

- A hash maps arbitrary data to a **fixed-length output**
- Used to detect **data modification**
- Even a small change produces a completely different hash

Properties

- **Collision resistance**: it is **computationally infeasible** to find two different inputs $x \neq y$ such that $H(x) = H(y)$
 - not impossible, but would require $\approx 2^{128}$ operations for SHA-256 (birthday paradox)
- **Preimage resistance**: given a hash value $H(x)$, it is **computationally infeasible** to find the original input x
 - best known method is brute force and it requires $\approx 2^{256}$ attempts for SHA-256

Use Case

- verify message integrity
- detect tampering

Hash Key Operations

Key Operations

- `Hash(data)`: computes SHA-256 hash and returns it's hex representation
- `HashString(s)`: convenience wrapper for strings
- `HashMessage(...)`:
 - hashes logical message content and returns it
- `VerifyHash(data, expected)`:
 - recompute hash
 - compare with expected value

Important

Hash alone does **NOT** prove who sent the message.

Message Authentication Codes (crypto/mac.go)

- A **MAC** authenticates a message using a **shared secret key**.
- Only processes that know the key can:
 - create a valid MAC tag
 - verify a received message
- Protects both:
 - **Integrity** (message was not modified)
 - **Authentication** (who sent the message)

Properties (Cachin et al.)

- **Unforgeability**: without the key, valid tags cannot be created
- **Verification**: with the key, correctness can be checked

Key Idea

MAC = **Hash** + **Secret Key**

Only trusted processes can generate valid messages.

MAC Key Operations

Key Operations

- `ComputeMAC(key, data)`:
 - computes tag $t = \text{MAC}(k, m)$
 - uses HMAC-SHA256
- `VerifyMAC(key, data, tag)`:
 - recomputes expected MAC
 - compares using **constant-time comparison**

Behavior in Simulator

- Sender attaches MAC to message
- Receiver verifies MAC before accepting
- If verification fails:
 - message is **rejected**

Why not just use a hash?

- Hashes provide only **integrity**
 - anyone can compute a new hash
- Attacker can:
 - modify message
 - recompute hash
- No way to verify **who created the message**

Key Insight

MAC adds a **secret key**, so attacker cannot recompute valid tag.

Digital Signatures (crypto/signature.go)

- Digital signatures provide **non-repudiation**.
- A signer uses a **private key** to create a signature.
- Others use the corresponding **public key** to verify it.

What do signatures guarantee?

- **Authenticity**: message came from the claimed signer
- **Integrity**: message was not modified
- **Non-repudiation**: signer cannot later deny having signed it

Important Simplification

In a real system, signatures use **asymmetric cryptography** (e.g. RSA, Ed25519, ECDSA).
In this simulator, HMAC is used only as a **teaching stand-in** to model the abstraction and API.

Signatures vs MAC

MAC

- uses a **shared secret key**
- both sides can compute and verify
- does **not** provide non-repudiation

Digital Signature

- uses a **private/public key pair**
- only signer creates signature
- everyone with the public key can verify
- provides **non-repudiation**

Key Idea

MAC proves **shared-key authenticity**. Signature proves **who exactly signed the message**.

Key structures

```
type KeyPair struct {
    PublicKey  string
    PrivateKey string
}

type KeyRegistry struct {
    keys          map[string]KeyPair
    sharedKeys    map[string]string
}
```

- KeyPair stores a process's public/private keys
- KeyRegistry acts like a simplified **PKI**
- It manages:
 - per-process key pairs (for signatures)
 - pairwise shared keys (for MACs)

Signature Key Operations

Key Operations

- `RegisterProcess(processID):`
 - creates and stores a key pair for a process
- `Sign(processID, data):`
 - produces a signature using the process's private key
- `Verify(processID, data, sig):`
 - checks whether the signature is valid
- `GetPublicKey(processID):`
 - returns the public key of a process

Behavior in Simulator

- only the key holder can create valid signatures
- verification fails if the data was tampered with
- everyone with access to the registry can verify

Safety vs Liveness

Safety Property

- *Bad things never happen*
- Violations can be detected by observing a **single execution step**

Liveness Property

- *Good things eventually happen*
- Cannot be falsified by **finite execution**

Key Idea

Safety violations appear in a **finite prefix of a trace**. Liveness violations require **infinite observation**.